TECHNICKÁ UNIVERZITA V KOŠICIACH
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Flora Guard – Detekcia invazívnych druhov rastlín z fotografie

Zadanie č.2

2026

MELNÍK DANIEL, PRACKOVÁ DANIELA, ŠPIŠÁKOVÁ MÁRIA

Obsah

ZOZNAM OBRÁZKOV **3**

ÚVOD **4**

1. POUŽITÉ TECHNOLÓGIE **5**

1.1. FRONTEND 5

1.2. BACKEND 5

1.3. DATABÁZA 5

1.4. UMELÁ INTELIGENCIA 5

1.5. CLOUD PLATFORMY 5

2. ARCHITEKTÚRA APLIKÁCIE **6**

3. REALIZÁCIA PROJEKTU – MELNÍK DANIEL **7**

3.1. FRONTEND 7

3.2. BACKEND 7

3.3. AI SLUŽBA 7

REALIZÁCIA PROJEKTU – PRACKOVÁ DANIELA **8**

3.4. DATABÁZA – AZURE DATABASE FOR POSTGRESQL 8

3.5. AZURE BLOB STORAGE – UKLADANIE OBRÁZKOV 8

3.6. BACKEND – ROZŠÍRENIE FLASK APLIKÁCIE 8

3.7. FRONTEND – OPRAVA HISTÓRIE 9

REALIZÁCIA PROJEKTU – ŠPIŠÁKOVÁ MÁRIA **10**

3.8. ARCHITEKTÚRA FRONTENDU 10

3.9. KOMUNIKÁCIA S BACKENDOM A SPRACOVANIE VÝSLEDKOV 10

ZÁVER **12**

Zoznam Obrázkov

Obrázok 1 - Architektúra 6
Obrázok 2 Domovská stránka aplikácie 11
Obrázok 3 Informácie o projekte 11

Úvod

Cieľom projektu **Flora Guard** je vytvoriť dostupnú webovú aplikáciu, ktorá:

- Umožňuje používateľom **nahrať fotografiu rastliny** prostredníctvom intuitívneho rozhrania
- **Automaticky klasifikuje** rastlinu pomocou neurónovej siete a určí, či ide o invázny druh
- Poskytuje **odporúčanie** pre používateľa (nahlásiť alebo žiadna akcia)
- **Ukladá výsledky** do databázy pre budúcu analýzu a štatistiky

Aplikácia je navrhnutá ako multi-cloudová architektúra s dôrazom na škálovateľnosť, modularitu a oddelenie zodpovedností jednotlivých komponentov.

1. Použité technológie

1.1. Frontend

- **HTML5** — sémantická štruktúra dokumentu
- **CSS3** — responzívne štýly s eco-friendly dizajnom
- **JavaScript (ES6+)** — interaktivita, drag & drop, fetch API

1.2. Backend

- **Python 3.11** — programovací jazyk
- **Flask 3.0** — mikro-framework pre REST API
- **flask-cors** — riešenie cross-origin requestov
- **requests** — HTTP klient pre komunikáciu s AI službou
- **Gunicorn** — produkčný WSGI server

1.3. Databáza

- **PostgreSQL** — relačná databáza pre persistenciu výsledkov

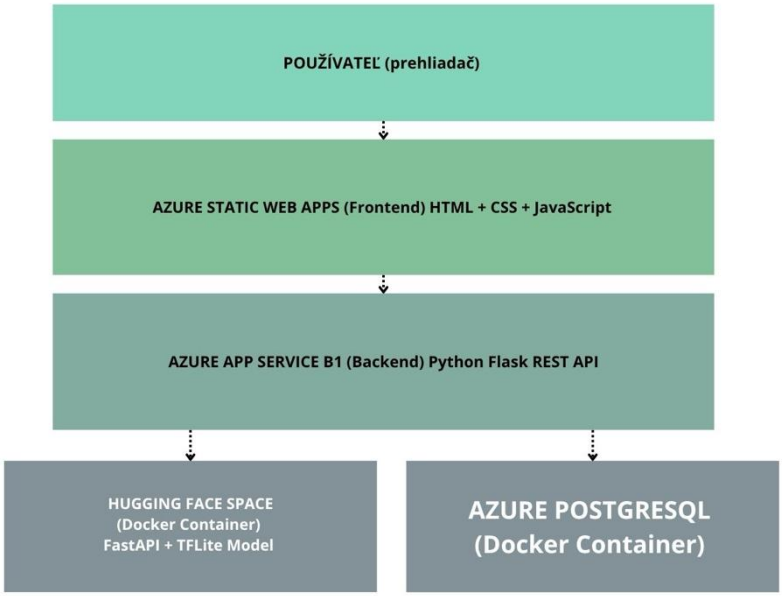
1.4. Umelá inteligencia

- **TensorFlow 2.x / Keras** — tréning neurónovej siete
- **MobileNetV2** — pre-trénovaná konvolučná sieť (transfer learning)
- **TensorFlow Lite** — komprimovaný model pre rýchlu inferenciu
- **FastAPI** — moderný Python framework pre AI API
- **NumPy** — spracovanie obrazu

1.5. Cloud platformy

- **Microsoft Azure** — hosting frontendu, backendu a databázy
    - Azure Static Web Apps
    - Azure App Service (Tier B1)
    - Azure Database for PostgreSQL
- **Hugging Face Spaces** — hosting AI služby (Docker container)

2. Architektúra aplikácie



Obrázok 1 - Architektúra

3. Realizácia projektu – Melník Daniel

3.1. Frontend

Frontend sa nasadzuje **automaticky cez GitHub Actions** pri každom push-e na hlavný branch repozitára. Azure Static Web Apps zabezpečuje globálnu CDN distribúciu a HTTPS certifikát.

3.2. Backend

Backend som realizoval v jazyku Python s frameworkom Flask. Aplikácia beží na Azure App Service v tieri B1 Basic, ktorý zabezpečuje konzistentný response time bez "spánku" služby.

API obsahuje štyri endpointy: koreňový endpoint s informáciami o službe, health check /api/ping, debug endpoint /api/info a hlavný endpoint /api/predict, ktorý prijíma fotku, preposiela ju na AI službu, spracováva odpoveď a vracia výsledok frontendu vo formáte JSON.

Pre fungovanie komunikácie medzi frontendom a backendom (rôzne domény) som nastavil CORS prostredníctvom knižnice flask-cors. Backend ďalej obsahuje robustný error handling — timeout pre cold start AI služby, retry logiku pri sieťových problémoch a informatívne chybové hlášky pre používateľa.

Deployment prebieha rovnako ako pri frontende — cez GitHub Actions automaticky po push-i.

3.3. AI služba

Pôvodne som AI komponent plánoval nasadiť na AWS Lambda, ale narazil som na technické komplikácie s GLIBC kompatibilitou TensorFlow runtime. Po niekoľkých neúspešných pokusoch som zvolil **Hugging Face Spaces s Docker runtime**, ktorý poskytuje plnú kontrolu nad prostredím a zároveň **splňa požiadavku zadania na nasadenie služby na inom cloude** ako zvyšok aplikácie.

AI službu tvorí **FastAPI** server bežiaci v Docker kontajneri. Endpoint `/predict` prijíma obrázok cez multipart formulár, preprocessuje ho podľa štandardu MobileNetV2.

Realizácia projektu – Pracková Daniela

3.4. Databáza – Azure Database for PostgreSQL

Na ukladanie výsledkov detekcií bola vytvorená relačná databáza na cloudovej platforme Microsoft Azure pomocou služby Azure Database for PostgreSQL Flexible Server (tier Burstable B1ms). Bola nakonfigurovaná verejná sieťová dostupnosť a pridaná IP adresa klienta do firewall pravidiel.

Bola vytvorená databáza **invasive_plants** s tabuľkou **detections**, ktorá uchováva výsledky každej detekcie: názov obrázka, URL obrázka, presnosť invázie, presnosť invázie v percentách, správa, odporúčanie, timestamp a IP adresa klienta.

3.5. Azure Blob Storage – Ukladanie obrázkov

Pre ukladanie nahratých fotografií rastlín bol vytvorený Storage Account **invasiveplantsstorage** s kontajnerom **detections** (úroveň prístupu: Blob – verejné čítanie). Každý obrázok je uložený pod unikátnym UUID názvom a jeho veľejná URL je zapsaná do databázy. Pripojovací reťazec bol nastavený ako environment variable na Azure App Service.

3.6. Backend – Rozšírenie Flask aplikácie

Súbor **backend/app.py** bol rozšírený o podporu databázy a Blob Storage. Boli pridané nasledujúce funkcie a endpointy:

get_db_connection() – vytvorí spojenie s PostgreSQL databázou cez environment variable DATABASE_URL; ak je databáza nedostupná, vráti None.

upload_image_to_blob() – nahraje obrázok do Azure Blob Storage, generuje UUID názov súboru a vráci veľejnú URL.

save_detection() – uloží výsledok detekcie vrátane URL obrázka do tabuľky detections v PostgreSQL.

GET /api/history – nový endpoint vracíajuci posledných 50 detekcií z databázy vrátane URL obrázkov.

GET /api/info – rozšírený o polia database_connected a storage_connected pre monitoring stavu služieb.

Do súboru **requirements.txt** boli pridané závislosti **psycopg2-binary** (PostgreSQL driver) a **azure-storage-blob** (Azure Blob Storage klient).

3.7. Frontend – Oprava histórie

Komponent **frontend/components/history.js** bol upravený v dvoch oblastiach. Po prvé, odstránena duplicita záznamov – história pôvodne kombinovala záznamy z databázy aj z localStorage prehliadača, čo spôsobovalo zobrazeníe každého záznamu dvakrát; blok zlučovania localStorage bol odstránený. Po druhé, bolo doplnené zobrazenie náhľadu obrázka – ak záznam obsahuje image_url, v histórii sa zobrazí skutočná fotografia rastliny; pre staršie záznamy bez URL zostáva emoji ako náhradný obsah. Zároveň sa pri načítaní stránky automaticky vyčistí stará lokálna história z localStorage.

Realizácia projektu – Špišáková Mária

3.8. Architektúra frontendu

Frontend aplikácie FloraGuard som navrhla a implementovala ako používateľské rozhranie, ktoré umožňuje jednoduchú interakciu s backendovým systémom na detekciu inváznych rastlín. Pri návrhu som sa zamerala najmä na prehľadnosť, intuitívne ovládanie a moderný vizuálny štýl, ktorý zodpovedá ekologickej téme projektu. Frontend som vytvorila pomocou HTML, CSS a JavaScriptu bez použitia frameworkov. Aplikácia funguje ako **Single Page Application (SPA)**, čo znamená, že všetky stránky (Domov, Detekcia, História, O projekte) sú súčasťou jedného HTML súboru a prepínajú sa dynamicky pomocou JavaScriptu bez potreby obnovenia stránky.

Projekt som rozdelila do viacerých častí pre lepšiu prehľadnosť a udržiavateľnosť:

- index.html – obsahuje štruktúru aplikácie a jednotlivé sekcie,
- style.css – zabezpečuje vizuálny vzhľad aplikácie,
- config.js – obsahuje konfiguráciu, najmä URL backendu,
- api.js – rieši komunikáciu s backendom,
- components/ – obsahuje logiku pre jednotlivé časti aplikácie (upload, výsledok, história, navigácia),
- script.js – riadi prepínanie stránok a základné správanie aplikácie.

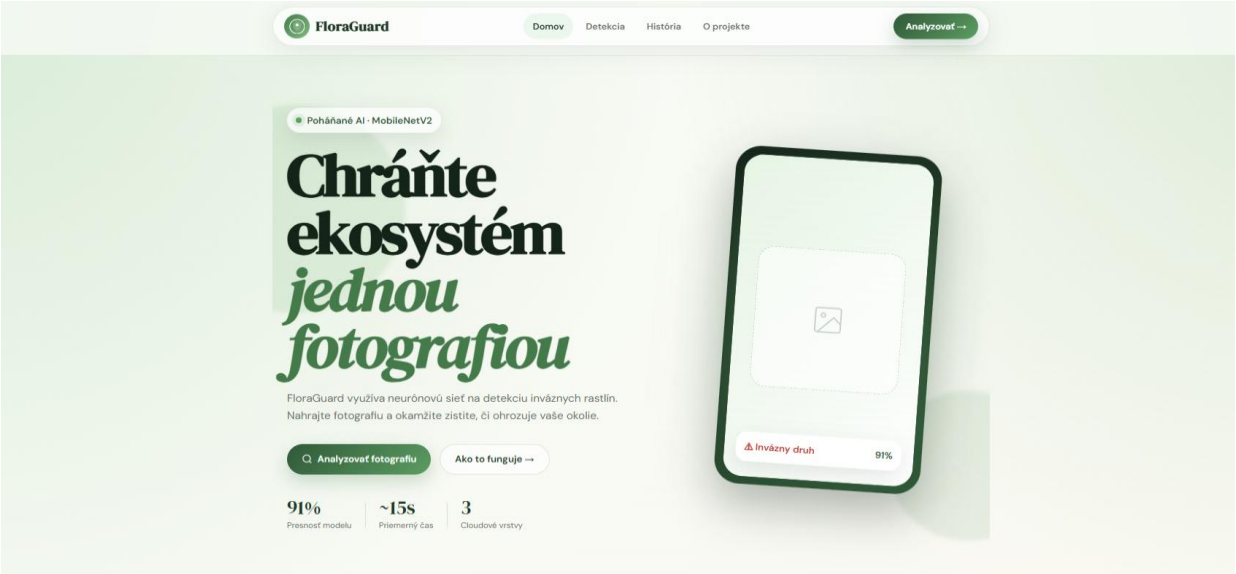
3.9. Komunikácia s backendom a spracovanie výsledkov

Frontend aplikácie komunikuje s backendom prostredníctvom REST API, pričom na realizáciu HTTP požiadaviek využíva rozhranie **Fetch API**. Jednotlivé API volania sú implementované v samostatnom module, čo zvyšuje prehľadnosť a modularitu kódu. Obrázok rastliny je odosielaný na endpoint /api/predict vo formáte **multipart/form-data**, pričom backend zabezpečuje jeho spracovanie pomocou AI modelu. Okrem toho frontend využíva aj ďalšie endpointy, napríklad /api/history na načítanie histórie detekcií a /api/ping na kontrolu dostupnosti backendu.

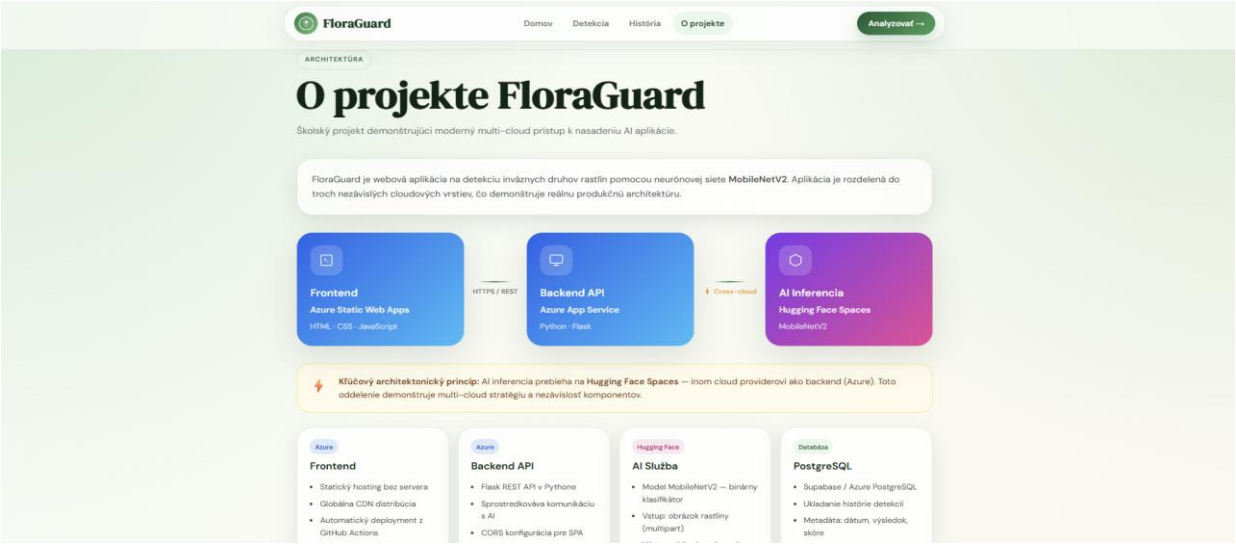
Po spracovaní obrázka backend vráti výsledok, ktorý frontend zobrazí vo forme výsledkovej karty. Tá obsahuje informáciu o tom, či ide o invázny druh, mieru istoty (confidence) a odporúčanie pre používateľa. Výsledok je vizuálne odlíšený farbou, aby bol pre používateľa ľahko interpretovateľný – červená farba označuje invázny druh, zelená neinvázny a oranžová signalizuje chybový stav.

Súčasťou aplikácie je aj sekcia histórie, v ktorej sa zobrazujú predchádzajúce detekcie. Tieto dáta frontend získava z backendu a dynamicky ich zobrazuje vo forme kariet, pričom každá položka obsahuje základné informácie o analýze, ako napríklad výsledok a mieru istoty.

Frontend zároveň zobrazuje aj stav jednotlivých častí systému, ako sú backend, databáza a AI služba. Tieto informácie sa aktualizujú na základe odpovedí z backendu a poskytujú používateľovi prehľad o aktuálnej dostupnosti a funkčnosti systému.



Obrázok 2 Domovská stránka aplikácie



Obrázok 3 Informácie o projekte

Záver

V rámci projektu FloraGuard sa nám podarilo vytvoriť funkčnú webovú aplikáciu, ktorá prepája frontend, backend a AI model do jedného celku. Jednotlivé časti systému spolupracujú prostredníctvom definovaného rozhrania, čo zabezpečuje plynulý prenos dát a správne spracovanie požiadaviek používateľa.

Aplikácia umožňuje používateľovi jednoducho nahrať obrázok rastliny, ktorý je následne analyzovaný pomocou AI modelu. Výsledok detekcie je prehľadne zobrazený spolu s mierou istoty a odporúčaním. Súčasťou riešenia je aj ukladanie a zobrazovanie histórie detekcií, ako aj základný prehľad o stave systému.

Pri implementácii sme využili technológie HTML, CSS a JavaScript na strane frontendu a Python s frameworkom Flask na strane backendu. AI model je nasadený samostatne, čo umožňuje oddelenie jednotlivých častí systému a jednoduchšie rozširovanie aplikácie v budúcnosti. Nasadenie riešenia do cloudového prostredia zabezpečuje jeho dostupnosť a praktické využitie.

Počas realizácie projektu sme získali praktické skúsenosti s návrhom webovej aplikácie, komunikáciou medzi frontendom a backendom, prácou s API a integráciou umelej inteligencie do reálneho riešenia. Zároveň sme si prehĺbili znalosti v oblasti nasadzovania aplikácií do cloudu a riešenia problémov pri prepojení jednotlivých komponentov.

Do budúcnosti vidíme priestor na ďalšie rozšírenie aplikácie, napríklad v oblasti zlepšenia presnosti modelu, rozšírenia databázy detekovaných druhov alebo doplnenia ďalších funkcionalít pre používateľov.